

Developer modes and security trust in AI generated code

A concise research proposal and working plan

Nimna Pathum | 4 October 2026

1 Aim and research value

Aim: understand how early-career developers judge, verify, and retain AI-generated code while pursuing a known implementation plan or exploring an uncertain approach, and how their security confidence aligns with the code’s assessed security.

The study is valid as a controlled investigation of human judgment during agent-assisted programming. Barke et al. identify acceleration and exploration as interaction modes [1]. Perry et al. show that security outcomes and developers’ security beliefs can diverge [2], while Sandoval et al. report a different pattern in a different task setting [3]. This makes the relationship an empirical question; the proposal should not predict that acceleration must be worse.

The contribution is the combination of mode-oriented conditions, episode-level mode evidence, repeated security judgments, and observed verification in two agent-assisted projects. Security review and confidence are already studied, including a recent study with 100 participants [4]. A defensible novelty claim is this specific combination and population, subject to a final literature update before submission.

2 Refined research questions

Question	Scope and intended answer
RQ1 Primary	How does alignment between developers’ security confidence and assessed code security differ between acceleration-oriented and exploration-oriented conditions? How does this relate to observed interaction mode?
RQ2 Secondary	Does an insecure neighboring implementation increase retention of an insecure AI-generated candidate, and does this relationship differ by task condition?
RQ3 Secondary	How do verification actions before and after provisional acceptance differ by condition, and how are they associated with vulnerabilities remaining at submission?
RQ4 Exploratory	Do these patterns differ between SQL injection and path traversal?

Success means estimating these relationships with uncertainty and explaining the observed decisions. A null or mixed result is useful. The study will not establish a universal mental-state detector, rank coding agents, or demonstrate that junior developers are inherently overtrusting.

3 Recommended study design

Use a mixed-method, within-participant study: each person completes exactly two main projects, one in each assigned condition, using JavaScript and Antigravity agent mode. Embed four short, controlled review checkpoints in each project. These are project features, not eight additional full programming tasks.

Comparable exposure while retaining live agent use

Begin each project with live agent-assisted inspection and setup. At each checkpoint, the study extension presents one prepared AI-origin patch generated with the study’s chosen agent and model. Participants can inspect it, ask the live agent questions, keep or reject it, and subsequently repair or replace it. Keep the presentation identical in both conditions. Tell participants that some suggestions were prepared in advance; never label a prepared patch as a response just produced by their live agent. Record generation provenance and researcher edits. Controlled suggestions have precedent in security studies [4,5]; this particular hybrid workflow needs a pilot.

Option	Decision and consequence
Natural generation only	Most natural interaction, but some participants may encounter no flaws. Report exposure and outcomes separately; never add failures after seeing a participant’s behavior. Suitable as a later replication.
Hidden Markdown instructions	May influence the agent, but cannot guarantee the requested flaw. Also changes the study toward manipulated agent context. Keep as an optional generation pilot, outside the main experiment [5].
Prepared candidate checkpoints	Recommended for the main study. Equalizes secure and insecure exposure. Conclusions apply to reviewing prepared AI-origin code within an agent-assisted workflow, not to the natural vulnerability rate of Antigravity.

Balance the experiment

Each project has two SQL and two file-handling checkpoints, with one secure and one vulnerable candidate per class. Across the four checkpoints, cross candidate security with secure versus insecure neighboring context once each. Rotate which class receives each combination across participants. Keep the candidate itself identical across its two context versions. The neighboring example is visible but outside the submitted target function and is not counted as a participant-created flaw.

Randomize balanced lists crossing project order and condition assignment: A acceleration then B exploration; B exploration then A acceleration; A exploration then B acceleration; B acceleration then A exploration. Rotate patch versions and checkpoint order within these lists. Keep the secure fraction undisclosed, give no correctness feedback between blocks, and examine period effects because learning can carry over. Each person therefore sees eight candidates, including four vulnerable exposures. Repeated observations do not turn 30 participants into 240 independent people.

RQ2 tests the influence of a visible code example on the developer’s decision. Agent imitation caused by hidden rules is a different mechanism and must not be described as the same experiment.

4 Tasks and mode preparation

Use one language to reduce language-skill variation: JavaScript on Node.js, a small local HTTP service, SQLite, and a temporary file store. Pin dependency versions. Include only SQL injection and path traversal in the main study; secret handling and transport security introduce different knowledge and infrastructure demands.

Project brief	Four independent feature checkpoints
A Resource catalogue "Complete the service so users can find catalogue items and retrieve stored documents."	1 Exact title lookup. 2 Category lookup. 3 Download a named attachment. 4 Preview a text note.
B Support archive "Complete the service so users can find tickets and retrieve stored support records."	1 Exact subject lookup. 2 Status lookup. 3 Download a named export. 4 Preview an archived note.

Provide equivalent schemas, route skeletons, input lengths, public functional tests, and documentation. Each target function should be short enough to inspect in a few minutes. Prepared patches touch separate functions; reset only the next untouched skeleton and preserve previous participant work. Use fictional data and disposable local files.

Encourage a mode without defining it by task difficulty

For acceleration, give a short practice example using that project's API adapter and ask the participant to describe their intended approach. For exploration, provide an equally documented but unrehearsed adapter and ask them to discover how to implement the features. Both adapters wrap the same underlying libraries and expose equally capable operations. Counterbalance which adapter is rehearsed; both projects can receive either condition.

This is an adaptation of the familiarity and uncertainty distinction in Grounded Copilot [1], not a validated induction. Do not make one project trivial and the other much harder. Give equal main-task time and the same security expectations. Familiarization can affect performance directly, so causal conclusions concern the assigned preparation condition. Observed mode supports an additional association, not a proven causal mediator.

Example security oracles

Class	Vulnerable and secure variants	Researcher checks
SQL injection	Unsafe: concatenate an untrusted title into SQL. Safe: bind it as a query parameter. Both pass ordinary lookups.	Known literal input succeeds; adversarial input cannot broaden results. Verify actual SQLite behavior and data flow [15].
Path traversal	Unsafe: use the requested name in a filesystem path without containment. Safe: enforce the documented file policy and resolved-path containment.	Valid file succeeds; parent traversal, absolute paths, and encoded traversal cannot expose an outside canary [16].

Specify URL decoding, operating system, and symlink policy before building tests. Disable symlinks in fixtures or test real-path containment. Reject "secure" variants that simply break required functionality. Two independent reviewers validate all variants before recruitment.

5 Participants and session procedure

Population and sample

Recruit final-year CS or IT students with substantial programming experience and junior engineers with up to two years of professional development experience. Require basic JavaScript competence. Record student versus employed status, months of programming, security education, JavaScript familiarity, and agent-use frequency. Entry-level status does not imply little AI experience. Developer sampling affects conclusions [6].

Plan 4–6 pilot participants, excluded from the main analysis, followed provisionally by 30 main participants. This is a feasibility target, not a power result. Before recruitment, simulate the planned paired analysis using plausible effect sizes, within-person dependence, missingness, and pilot timing. Increase the target if feasible; otherwise explicitly frame small interactions in RQ2 and RQ4 as exploratory.

Separate knowledge from reliance

Screen programming skill with a short supervised function-reading exercise, adapted from programmer-screening research [7]. Assess security knowledge before the session with a brief, separate questionnaire without feedback: two items each on parameterized queries and file containment, plus distractors on ordinary programming. Score explanations using a fixed rubric. Because this can prime security attention, describe the study as security-salient and use the same procedure for everyone.

After both tasks, present novel matched snippets without AI assistance and ask participants to identify and explain risks. This helps distinguish failure to recognize a risk from failure to act on known risks, but it is a post-task diagnostic that may reflect learning. It cannot prove that a person possessed the same knowledge at an earlier moment.

Stage	Action and record	Target time
Preparation	Consent, screen and knowledge measures; neutral IDE practice; recording and logging checks.	20 min
Task block 1	Condition preparation, four checkpoints, final snapshot, short mode interview.	40 min
Break	Fresh workspace and conversation for the next project.	5 min
Task block 2	Other condition; same checkpoint procedure and timing.	40 min
Close	Unaided diagnostic, comparative interview, suspicion check and full debrief.	20 min

Aim for about two hours; shorten after timing pilots if fatigue is evident. Avoid concurrent think-aloud in the main tasks because it can change checking behavior. Use screen-assisted retrospective interviews, supported by prior contextual trust and interaction studies [8,9].

6 Measurements and trust calibration

Keep three constructs separate: trust is an attitude toward the agent; reliance is a keep or reject decision; security confidence is a probability judgment about a specific code snapshot. An agent writing code into a file is an exposure, not evidence that the human accepted it [8,10,11].

A consistent checkpoint sequence

1 Show and archive the original candidate. 2 Let the participant inspect it or consult the agent, but keep this original candidate immutable. 3 Record “keep this version provisionally” or “reject this version”; replacements can be requested next. 4 Ask: “From 0 to 100%, how likely is this displayed version to satisfy the task’s security requirements?” Record a snapshot hash. 5 Allow unrestricted repair or replacement until feature completion. 6 At project submission, archive final code and collect confidence in each final target function.

Ask the candidate-confidence question after both keep and reject decisions, before further editing. This avoids measuring only accepted code. Explain 0, 50 and 100 using a neutral practice example. Rating prompts may trigger extra checking; label all subsequent verification as occurring after confidence elicitation and preserve earlier checks separately.

Measure	Definition and interpretation
Primary judgment score	Brier score = $\text{mean}((p - y)^2)$, where p = confidence / 100 and $y = 1$ if the candidate meets the prespecified security criterion, otherwise 0. Lower is better. Brier measures overall probability accuracy, including discrimination; it is not pure calibration [12].
Calibration description	By condition, report $\text{mean}(p) - \text{mean}(y)$, confidence distributions and a coarse reliability plot with uncertainty. Positive gap suggests average overconfidence. Opposing errors can cancel. Eight observations cannot establish a reliable individual calibration curve.
Unsafe provisional retention	Number of vulnerable candidates kept / number of vulnerable candidates displayed. Also report secure candidates rejected / secure candidates displayed; rejection alone does not prove distrust.
Final security and detection	Unresolved exposed flaws / vulnerable exposures; correctly articulated detections / vulnerable exposures; verified repairs / exposed flaws. Distinguish detection, silent removal, and agent-led repair.
Verification behavior	Human-requested tests or reviews, manual inspection evidence, adversarial tests, independent documentation, and agent-led checks. Record before keep, after keep, and after the confidence prompt.
Trust attitude	After each project, 1–7 agreement with “I could rely on this agent for this task” and “I trusted its security-related advice.” These are study-specific descriptive items, not a validated scale [8,10].

Worked example: confidence 80% in a vulnerable candidate gives $(0.8 - 0)^2 = 0.64$. Confidence 80% in a secure candidate gives 0.04. If four judgments average 75% but only 50% of candidates are secure, the average gap is +25 percentage points. Ground truth applies to the declared threat model, not all possible vulnerabilities.

7 IDE extension and data collection

Build a small local study extension plus a passive agent-log collector. CodeWatcher demonstrates useful editor telemetry but its small evaluation also shows imperfect AI-origin classification [13]. Do not treat large edits, fast insertion, inactivity, or clipboard activity as authoritative agent acceptance.

Data	Collection route	Limit or validation
Candidate and decision	Study extension owns the checkpoint UI; record display, keep, reject, confidence, candidate ID and code hash.	Direct events for controlled candidates. Capture final submission separately.
Editing and navigation	VS Code-compatible document change/save, active editor, selection and visible-range events; periodic versioned snapshots.	Verify APIs in the exact Antigravity build. Visible text does not prove reading or attention [14].
Prompts and responses	Use documented Antigravity transcript access exposed by hooks; export only study conversations.	Pilot the schema. Another extension's private chat is not generally exposed by the VS Code chat API [14,17].
Agent actions	Passive PostToolUse, PostInvocation and Stop hooks; link conversation ID, timestamps and transcript path.	Tool events may not contain full output. Check transcript completeness. Do not override permission decisions [17].
Tests and diagnostics	Instrument the study test runner; capture command, initiator, result and snapshot. Archive diagnostics.	Use supported terminal shell integration where available. Screen review covers uninstrumented commands.
Acceptance in live work	Explicit participant checkpoint and final submit actions, supported by video and version history.	Native agent auto-application and mixed conversation diffs do not independently establish human acceptance [18].

Minimum event record

Store participant_id, session_id, task_id, checkpoint_id, assigned_condition, event_id, timestamp_utc, elapsed_ms, source, event_type, actor, conversation_id, file_path, snapshot_hash, and payload. Actor values are human, agent, study system, or unknown. Store confidence, choice, patch version and context assignment as separate fields; append security labels only after scoring.

Synchronize logs and screen recording with a visible start marker. Use task-scoped JSONL, a manifest of tool and model versions, and hashes of snapshots. Export a participant table, checkpoint table and behavior-event table for analysis. Missing recordings or failed hooks must be marked as missing, not “no verification.”

Before recruitment, replay scripted edit, reject, keep, test, agent-repair and restart sequences against video. Require every checkpoint decision and confidence response to be recoverable. Measure precision and recall for inferred events; manually code any unreliable category. Antigravity plans and quotas can change, so test capacity and record the exact build, model and settings rather than assuming free access is sufficient [19].

8 Mode evidence and interview guide

Use acceleration and exploration as episode-level intentions [1]. Acceleration means pursuing an already understood approach with agent help. Exploration means using the agent to discover an approach or understand unfamiliar behavior. Allow mixed and unclear labels, and changes within a task. These are not diagnoses of System 1, System 2, attention or a hidden mental state.

Immediately after each project, replay one keep decision and one revise or reject episode, where available. Ask open questions first, then the probes below. The wording is newly adapted for this study; its theoretical grounding is not equivalent to questionnaire validation. Pilot comprehension and neutral wording.

Question to ask	Construct and source basis
Before the agent responded here, what did you already know you wanted to do?	Known plan versus unresolved approach; Grounded Copilot [1].
What were you asking the agent to help you achieve at this point?	Implementing a plan, discovering options, understanding an API, or another goal [1,9].
Did your goal or understanding change during this episode? Show where.	Mode switching; avoid imposing one label on a whole project [1,9].
What made you keep, reject, or change this version?	Contextual trust and decision cues [8,10].
What evidence did you use to judge its security? What remained uncertain?	Separate perceived trustworthiness from supporting evidence [8,11].
Which checks were your decision, and which did the agent initiate? What did you learn from them?	Agency and verification; artifact-supported reconstruction [4,9].
How, if at all, did the nearby example influence your decision?	Study-specific RQ2 probe, grounded in contextual trust [8,10]. Ask after free recall to reduce suggestion.
At this moment, were you mainly implementing a known approach, finding an approach, both, or neither? Why?	Participant interpretation after open questions; adapted from [1], not a validated mode scale.

Coding and triangulation

Two researchers independently label all eight checkpoint episodes using intention statements, relevant prompts, and video context. Code verification separately. Do not label mode from acceptance speed or lack of testing and then use those same behaviors as evidence of a mode effect. Keep security outcome and confidence scores hidden from mode coders where practical.

Pilot the codebook, report agreement and Cohen's kappa with uncertainty, then resolve disagreements while preserving original labels. Analyze mixed and unclear cases explicitly. If assigned conditions do not change reported intent or observed modes, report a preparation-condition study and revise the mode claim. Telemetry supports the reconstruction; it cannot confirm a participant's mind state.

9 Evaluation and analysis

Establish security outcomes independently

For each original candidate, provisional state and final function, run functional tests and targeted security tests in a disposable environment. Two reviewers inspect the relevant data flow and reconcile disagreements against a written rubric. Score secure, vulnerable, or indeterminate within the stated scope. A broken or unreachable endpoint is a functional failure, not a successful secure repair. Retain unresolved execution cases as indeterminate. Track additional flaws introduced during live agent work separately, with actor and snapshot provenance; do not add them to the controlled-exposure denominator.

Use CodeQL, Snyk Code or SonarQube as a supporting check after submission, with versions and rules recorded [15,16,20]. Dependency-vulnerability scanning alone cannot evaluate the custom injection and path logic. A scanner warning needs confirmation; a clean scan does not establish security. Keep researcher scans and hidden tests outside the agent-accessible workspace. Participant-initiated scans remain permitted and logged.

Prespecified comparisons

Question	Analysis and output
RQ1	Primary: each person’s mean candidate Brier score in each assigned condition; estimate the paired difference with a participant-level bootstrap confidence interval and paired randomization test matching the allocation. Show calibration gap and reliability descriptively. Secondary model relates observed mode to scores; causal language is limited to assignment.
RQ2	Among vulnerable candidates, compare retention under insecure versus secure context, accounting for participant and patch version. Treat the context-by-condition interaction as exploratory. Report counts, risk differences and uncertainty; use a mixed logistic model only if data support it.
RQ3	Compare human-led checks by condition and stage. Relate checks to unresolved flaws among exposed cases. Distinguish detection before acceptance, later repair, and autonomous agent repair. Associations do not prove that checking caused an outcome.
RQ4	Present class-specific estimates and uncertainty, with an exploratory condition-by-class interaction. No claim of class differences from one significant result and one nonsignificant result.

Record project, period, patch version, prior security knowledge and agent experience. Use parsimonious adjusted sensitivity analyses; 30 participants cannot sustain many predictors. If binary models separate or fail, report participant-level paired summaries instead of forcing convergence. One primary contrast is confirmatory; clearly label the remaining comparisons exploratory or adjust a prespecified secondary family.

Analyze randomized assignment first, including mode noncompliance. Report completed exposures, missing confidence and indeterminate security labels by condition. Exclude only predeclared eligibility failures, withdrawal, or unrecoverable measurements at the relevant analysis level; do not exclude people for missing vulnerabilities. Compare complete-case results with transparent bounds where missingness could change conclusions.

Integrate quantitative results with interview explanations in a compact evidence table: observed action, stated reason, security outcome, and alternative explanation. Include contrary cases such as careful acceleration and uncritical exploration.

10 Detailed methodology flow

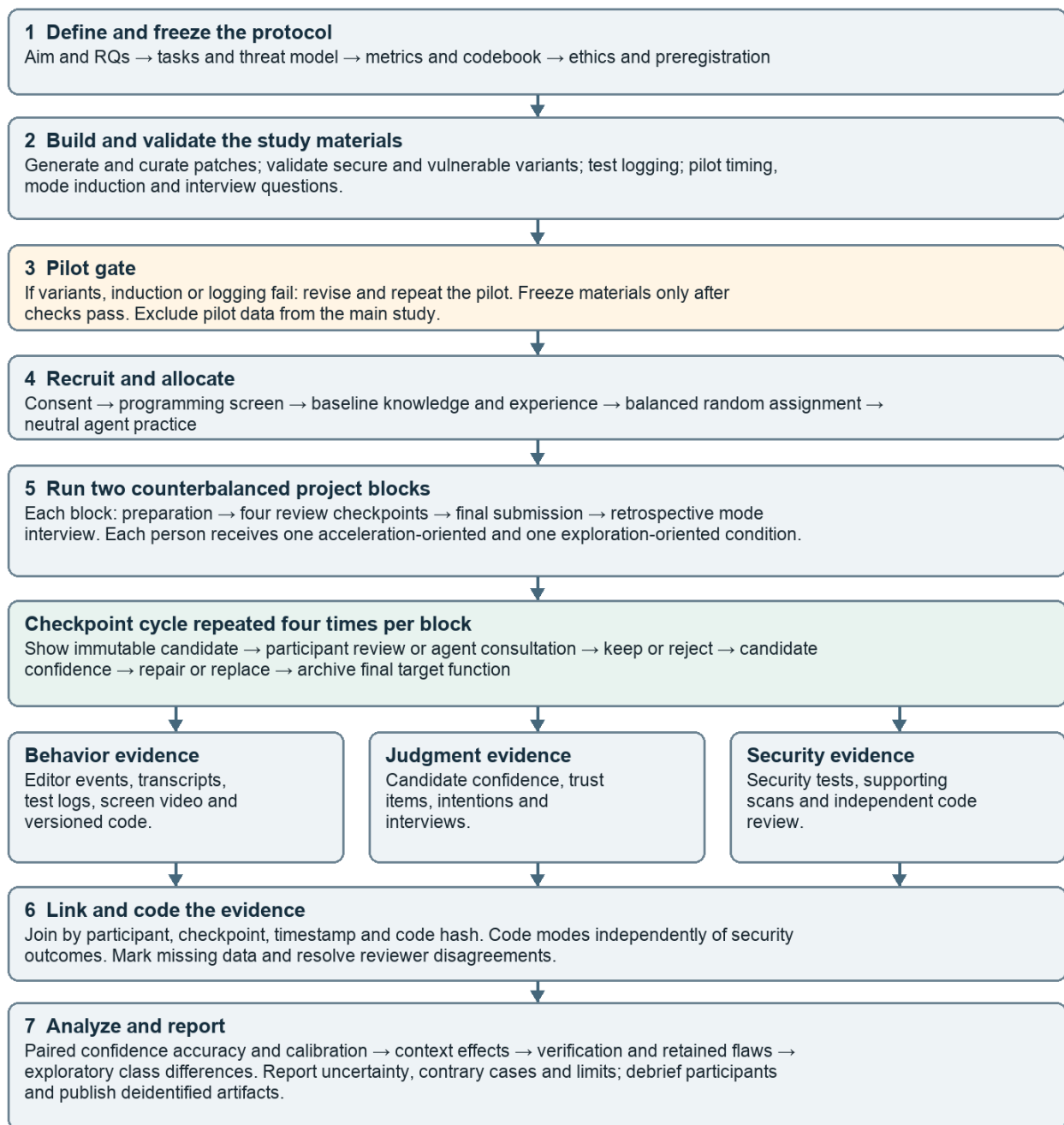


Figure 1. End-to-end study procedure. The branches represent complementary evidence, not three separate participant groups. Outcome labels are joined only after independent coding.

Keep all candidate IDs and snapshot hashes stable across the three evidence streams. Schedule debriefing immediately after the session; do not wait for the final analysis.

11 Validity safeguards and build checklist

How to defend the study

Likely challenge	Defensible response
Juniors simply lack security knowledge	Measure knowledge and experience, use within-person comparisons, and examine unaided recognition. If knowledge explains the pattern, report that result. Do not call every insecure decision overtrust.
Are these really cognitive modes	Use intent and retrospective evidence, allow switching, and test whether preparation changed mode distribution. Familiarity is an intervention with direct effects; a pure mental-state effect is not identified.
The vulnerabilities are artificial	Controlled exposure answers a behavioral question. Preserve realistic code and blind outcome assessment. Report prepared-candidate provenance and follow later with natural generation.
Checking is caused by the questionnaire	Separate checks before and after elicitation; acknowledge measurement reactivity and security priming. No covert claim of everyday unobserved behavior.
Two tasks are too few	Two projects contain eight judgments, useful for group contrasts. Individual calibration and broad class generalization remain unsupported.
The result was already established	Compare explicitly with [4]. The proposed advance is mode-linked evidence in agent-assisted JavaScript work, not the discovery that AI code can be insecure.

Markdown files and ethics

Give participants README.md and API_GUIDE.md, plus ordinary agent rules for scope and testing. They may read all task instructions. Do not ask them to attach a file they are forbidden to inspect: this can signal the manipulation and suppress a real verification behavior. Keep the variant manifest, answer key and hidden tests outside the workspace. Deliberate insecure-agent instructions belong only in a separate, ethically approved manipulation study [5].

Obtain institutional review for prepared insecure suggestions, recording, and any partial disclosure. Consent should describe AI-assisted programming, potentially flawed materials, data capture and withdrawal, while withholding exact flaw locations if approved. Debrief on manipulations and secure alternatives; provide a defined data-withdrawal window. Use study accounts and repositories, encrypted storage, a separate identity key, and institution-approved retention. Compensation must not depend on finding flaws.

Artifacts required before the main study

Build: two versioned task repositories; candidate and context manifest; generation provenance; functional and security test suite; checkpoint extension and passive logger; consent and debrief forms; screening and confidence forms; interview guide and coding rubric; randomization schedule; and an analysis script. Freeze these after the pilot. These are implementation deliverables, not completed instruments supplied by this proposal.

Compared with the initial proposal, this plan narrows broad SE correctness to two security classes, removes senior-developer comparisons and a mental-state classifier, replaces acceptance heuristics with explicit decisions, and treats confidence accuracy separately from trust attitudes. Prior research justifies the design choices; the pilot must validate this specific combination.

12 References and reusable resources

The numbered sources below support the design decisions. Adapted tasks, questions and measures still require piloting. Publication status is distinguished where relevant. Official tool documentation was checked on 4 October 2026.

- [1] Barke, S., James, M. B., and Polikarpova, N. (2023). Grounded Copilot: How Programmers Interact with Code-Generating Models. PACMPL, OOPSLA. Supplied 2022 preprint used for the interaction framework. [Source](#)
- [2] Perry, N., Srivastava, M., Kumar, D., and Boneh, D. (2023). Do Users Write More Insecure Code with AI Assistants? ACM CCS. Security outcomes and participant beliefs. [Source](#)
- [3] Sandoval, G., et al. (2023). Lost at C: A User Study on the Security Implications of Large Language Model Code Assistants. USENIX Security. Task-dependent security evidence. [Source](#)
- [4] Khalid, H., et al. (2026). (Don't) Trust, but (Don't) Verify: Developers' Attention to Security in AI-Generated Code. arXiv:2609.21020; listed as forthcoming at IEEE S&P 2027. Closely related controlled review study. [Source](#)
- [5] Oh, S., et al. (2024). Poisoned ChatGPT Finds Work for Idle Hands: Exploring Developers' Coding Practices with Insecure Suggestions from Poisoned AI Models. IEEE S&P. Controlled insecure-assistant precedent. [Source](#)
- [6] Kaur, H., et al. (2022). Where to Recruit for Security Development Studies: Comparing Six Software Developer Samples. USENIX Security. Recruitment and sample validity. [Source](#)
- [7] Danilova, A., et al. (2021). Do You Really Code? Designing and Evaluating Screening Questions for Online Surveys with Programmers. ICSE. Basis for a programming eligibility screen. [Source](#)
- [8] Wang, R., Cheng, R., Ford, D., and Zimmermann, T. (2024). Investigating and Designing for Trust in AI-powered Code Generation Tools. FAccT. Contextual trust and retrospective interviewing. [Source](#)
- [9] Wu, Y., et al. (2026). How Do Developers Interact with AI? An Exploratory Study on Modeling Developer Programming Behavior. FSE / arXiv:2604.16393. Screen-assisted reconstruction of intent and behavior. [Source](#)
- [10] Brown, A., et al. (2024). Identifying the Factors That Influence Trust in AI Code Completion. Alware. Suggestion, developer and context factors; limits of acceptance as a trust proxy. [Source](#)
- [11] Okamura, K., and Yamada, S. (2020). Adaptive Trust Calibration for Human–AI Collaboration. PLOS ONE, 15(2), e0229132. Calibration and reliance framework in a non-coding setting. [Source](#)

References and resources continued

- [12] Spiess, C., et al. (2024). Calibration and Correctness of Language Models for Code. Supplied paper. Supports probability-scoring concepts; studies model confidence, not a validated human trust questionnaire. [Source](#)
- [13] Basha, M., et al. (2025). CodeWatcher: IDE Telemetry Data Extraction Tool for Understanding Coding Interactions with LLMs. arXiv:2510.11536. Telemetry architecture and inference limitations. [Source](#)
- [14] Microsoft. VS Code Extension API. Document, editor, task, terminal and chat APIs; verify compatibility with the installed IDE. [Source](#)
- [15] GitHub. CodeQL query help: Database query built from user-controlled sources. JavaScript SQL-injection rule and examples. [Source](#)
- [16] GitHub. CodeQL query help: Uncontrolled data used in path expression. JavaScript path-injection rule and examples. [Source](#)
- [17] Google. Antigravity Hooks. IDE configuration, events and transcript-path fields. Implementation capability must be verified in a pilot. [Source](#)
- [18] Google. Antigravity Review Changes. Conversation review interface and scope of displayed changes. [Source](#)
- [19] Google. Antigravity Plans. Access and quota constraints for scheduling participant sessions. [Source](#)
- [20] SonarSource. Security-related rules. Distinguishes security findings and review needs; use as supporting evidence. [Source](#)

Useful starting artifacts

Use the Wu et al. replication package for examples of an annotation tool, experimental setup and surveys; adapt the structure, not its conclusions. Use the Kaur et al. study materials for recruitment and sampling documentation. Check permissions and licenses before reusing instruments or code.

Developer interaction replication package: [GitHub repository](#)

Security developer sampling materials: [Research data repository](#)

Supporting supplied literature includes Borg's 2024 position paper on IDE trust calibration and Parasuraman and Manzey's 2010 review of automation complacency and bias. These motivate the problem; neither validates an IDE-based diagnosis of a developer's cognitive state.